

中断、DMA 与操作系统 I/O

硬件异步完成怎样跨过软硬件边界，变成内核可确认的软件完成

408 · 跨科桥梁 X-B03

1 本册定位：只拥有软硬件完成交接

本册不是第二本 I/O 教材。它只回答一个接口问题：

母问题

设备可以脱离当前 CPU 指令流独立推进。那么当一次设备请求真正取得结果以后，什么硬件状态能够证明它已经完成，CPU/内核怎样发现这份证据，又怎样把它转换成软件可见的 request completion ？

责任边界固定为：

- 使用 CO-08：设备控制器、总线事务、中断入口、DMA 搬运及完成状态怎样由硬件形成；
- 使用 OS-05：驱动、I/O request、缓冲、阻塞语义和软件完成处理；
- 使用 OS-B01：等待者怎样从 Blocked 被解除等待并进入 Ready/Runnable；
- 本 Bridge 独立拥有：提交状态如何成为硬件输入；完成证据怎样被发现、核验并交回 OS；软硬件交接期间必须保持哪些合法性不变量；
- 停止边界：不重新讲轮询/中断/DMA/通道的完整机制，不展开磁盘调度、文件块映射、系统调用全过程，也不重新定义进程调度。

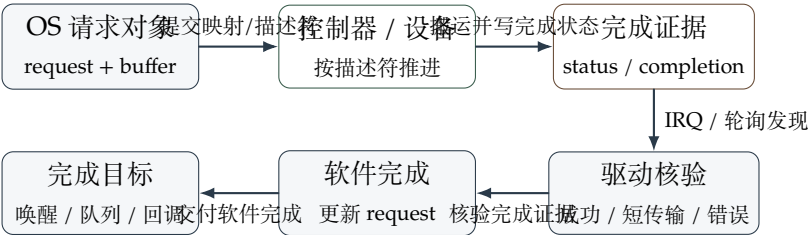
2 接口对象：不要把“一个 I/O”压成一个动作

一次跨界 I/O 至少涉及五类不同对象：

对象	它保存什么	主要 Owner
I/O request	本次请求的目标、方向、长度、缓冲位置及软件完成状态	OS-05
设备可见映射/描述符	设备在本次传输生命周期内能够使用的地址与传输描述	CO-08 与 OS-05 的交接口
数据缓冲区	设备与内存之间实际交换的数据，以及提交前后访问资格	OS-05 管理，硬件按描述符访问
完成证据	控制器状态位、完成队列项、错误码等“这次请求推进到什么状态”的硬件可观察事实	CO-08 产生，驱动消费
等待者/完成目标	阻塞线程、完成队列、回调或其他软件接收者	OS-05 / OS-B01

这里最重要的对象—表示边界是：“中断信号”不是“完成结果”本身。中断只是一种让 CPU 获得处理机会的通知形式；真正的软件完成必须能够追溯到与本次 request 对应的完成证据。

3 母接口：提交、推进、证明、发现、交付



图中所有实线箭头都表示接口交接；状态机内部的因果或逻辑推导不使用这组箭头。若只记这张图，应能从“完成证据”这个中点分别向左复原硬件怎样产生它、向右复原 OS 怎样消费它。

3.1 为什么必须按这个顺序

驱动不能在设备尚未获得合法描述符时假定它知道该搬什么；设备也不能只凭“数据已经搬了一部分”就让 OS 宣布 request 成功。稳定顺序来自责任约束：

1. OS 先建立本次请求及设备可消费的传输描述；

2. 设备/控制器只按这份描述推进数据与硬件状态；

3. 硬件形成与本次请求对应的完成证据；

4. CPU/驱动通过中断或轮询发现该证据；

5. 驱动核验成功、短传输或错误，再更新软件 request；

6. 只有软件完成已经成立以后，才有资格唤醒等待者或交付异步完成。

3.2 知识映射与 Owner 去向

母接口位置	深入时回到哪里	典型题面追问
提交与 DMA 描述	CO-08 + OS-05	DMA 地址、传送方式、描述符、缓冲区
硬件推进与完成证据	CO-08	控制器状态、中断请求、完成队列、总线/DMA 时序
发现与驱动核验	本 Bridge + OS-05	“中断到了是否等于完成”“轮询能否发现完成”
Blocked 到 Ready	OS-B01 / 调度专题	唤醒后为何不一定立即运行
完整 ‘read()’ 前后文	OS-I01 / X-I02 综合专题	文件对象、页缓存、I/O、调度怎样接成完整生命周期

这张表就是本册的章节/考纲映射：当前问题若继续向某一侧深入，立即回到对应 Owner，而不是让 Bridge 扩张成第二本 Topic。

4 接口为什么会出现：从朴素等待到异步交接

这一节采用 Problem、Naive Solution、Failure、Mechanism 的解释顺序；它描述设计动机，不表示四项之间存在数学蕴含关系。

4.1 问题

设备完成时间通常不与当前 CPU 指令流同步。系统既要让设备继续工作，又要在结果真正可用时恢复软件语义。

4.2 朴素方案

最直接的办法是让 CPU 不断读取设备状态，发现设备就绪后再由 CPU 指令完成后续处理。这个方案在简单场景中完全可以正确。

4.3 失败点

当等待时间远大于一次状态检查、或数据搬运量很大时，CPU 会把大量时间花在“重复确认还没完成”或逐单位搬运上。问题不在正确性，而在等待和搬运的 CPU 占用成本。

4.4 接口机制

硬件可以把“持续搬运”交给 DMA/设备，把“何时的事件需要软件处理”交给中断或完成队列；但这两个机制仍必须在一个明确的交接点重新汇合：**设备形成完成证据，驱动核验并把它转换成软件完成。**

因此本 Bridge 的生成性结论不是“DMA 比中断高级”，而是：

异步硬件要被软件可靠使用 ⇒ 必须存在可核验的完成交接

这里的 ⇒ 是当前接口目标下的必要设计约束：没有可识别、可归属到具体 request 的完成状态，OS 就无法可靠决定“谁已经完成、谁可以继续”。

5 生命周期：一个阻塞式设备读取怎样跨过接口

下面只跑软硬件交接段，不重讲完整 ‘read()’ 生命周期。

1. **请求形成**：上层已经判断本次读取需要真实设备 I/O，OS-05 建立一个 I/O request，并准备目标缓冲区。
2. **提交**：驱动建立本次 DMA 可见映射/描述符，写入方向、长度和设备命令。此时设备获得按协议访问目标缓冲区的资格。
3. **等待**：若用户接口采用阻塞语义且结果尚不可得，OS-05/OS-B01 使调用线程进入等待关系。状态变化应写成

Running $\xrightarrow{\text{等待 I/O 完成}}$ Blocked.

4. **硬件推进**：设备/DMA 按描述符在设备与内存之间搬运数据，并更新控制器内部状态。CPU 可以执行其他工作，但 DMA 仍可能占用内存/互连资源。
5. **形成完成证据**：控制器写入完成队列项或状态字段；如果请求失败，证据中还应包含可供驱动区分的错误状态。
6. **发现并核验**：CPU 可以因 IRQ 进入完成处理，也可以由软件轮询完成队列。驱动读取并消费与本 request 对应的完成证据，确认成功、短传输或错误。
7. **软件完成**：只有核验通过以后，request 才被更新为软件可见的完成状态。若存在阻塞等待者，则

Blocked $\xrightarrow{\text{request 完成并唤醒}}$ Ready/Runnable.

8. **停止在 Bridge 边界**：Ready/Runnable 以后何时真正 Running，由调度 Owner 决定；系统调用余下的复制、返回与文件状态更新由 OS Integration/Topic 继续负责。

6 接口不变量：哪些状态绝不能被破坏

1. **请求身份不变量**：完成证据必须能够归属到正确的 request；不能把“有一个中断”直接当成“当前等待的请求成功”。
2. **映射有效性不变量**：设备在传输生命周期内使用的 DMA-visible mapping 必须持续指向预期缓冲区；相关内存不能被错误回收并改作别用。
3. **缓冲区所有权不变量**：CPU 与设备何时可以读写缓冲区必须符合传输方向和平台一致性协议，不能产生破坏本次 I/O 语义的并发修改。
4. **完成资格不变量**：只有驱动消费并核验完成证据以后，软件 request 才能被标为完成。
5. **调度边界不变量**：I/O 完成最多直接解除等待；它不能单独推出线程已经获得 CPU。稳定状态关系是

Blocked $\xrightarrow{\text{completion/wakeup}}$ Ready 而不是 Blocked $\xrightarrow{\text{completion}}$ Running.

7 代价与权衡：交接越解耦，状态管理越多

本 Bridge 不比较哪一种 I/O 控制方式“最好”，只说明跨界接口的代价来源：

- **轮询发现完成**：减少中断进入/返回成本，但持续占用 CPU 检查时间；适合何种时间尺度由 CO-08/OS-05 的具体成本模型决定。
- **中断发现完成**：避免持续忙等，但要支付中断入口、完成处理以及可能的调度成本；事件过密时还会产生较高通知开销。
- **DMA 搬运**：减少 CPU 逐单位搬运，却引入描述符建立、DMA 映射、缓存一致性、互连争用和完成回收等状态成本。
- **批量完成**：可以摊薄每次通知成本，但通常会增加单个请求等待被批量处理的延迟。

因此稳定的 Tradeoff 不是“技术越新越快”，而是：**减少 CPU 对慢设备的细粒度参与，换来更多硬件状态、映射、队列和完成协议。**

8 概念边界：相似词不能互相推出

概念 A	≠	概念 B	为什么容易混、真正判据与题目信号	混淆后果
DMA 完成	≠	中断到达	两者常相邻发生。DMA 完成描述传输/request 状态已经形成完成证据；中断只是一种让 CPU 发现设备事件的入口。题目若给 completion queue/polling，就更不能把二者画等号	会把“结果已产生”和“CPU 已收到通知”压成同一时刻，写错事件顺序
中断驱动	≠	异步 I/O	都含“异步”直觉。前者回答设备事件怎样被 CPU 发现；后者回答调用返回与结果交付的 API 语义。阻塞‘read()’也可以由中断驱动设备完成	会由“用了中断”错误推出“调用线程立即返回”
DMA	≠	零拷贝	都减少某类 CPU 数据搬运。DMA 只说明设备与内存之间的搬运主体；用户/内核之间是否还复制要看完整数据路径	会漏算用户缓冲复制、映射与一致性成本
I/O 完成	≠	进程运行	完成处理可能唤醒 waiter，但 CPU 是否交给它仍由调度器决定。题目出现“中断完成后进程状态”时，应先写 Blocked 到 Ready	会把唤醒误写成直接调度，破坏进程状态机
DMA 地址	≠	用户虚拟地址	用户 VA 是进程地址空间表示；设备需要可访问的 DMA-visible mapping。经典题可按题设直接给物理主存地址，现代平台还可能经过 IOMMU	会把用户地址直接交给设备，混淆地址空间与访问主体

9 最小题面调用：什么时候打开这座 Bridge

只有题目同时跨过“硬件完成”和“OS 软件状态”时，才调用 X-B03。第一动作依次问：

1. 请求对象是谁？哪个 request、哪个缓冲区、方向和长度是什么？
2. 硬件怎样推进？数据主要由 CPU/PIO 还是 DMA/设备搬运？
3. 什么证明完成？状态寄存器、完成队列项还是其他 completion evidence？
4. CPU 怎样发现？IRQ 还是 polling？发现以后由谁核验？
5. 软件哪一层改变？request 完成、Blocked 到 Ready、完成队列/回调，还是尚未发生任何用户可见返回？

如果题目只问 DMA 控制器怎样传一块数据，停在 CO-08；如果只问阻塞线程怎样被唤醒，停在 OS-05/OS-B01；只有两边真正交接时才进入本 Bridge。

10 贯穿母例：中断到了，但请求并没有成功

设一个阻塞式设备读取已经提交，设备随后产生 IRQ。朴素判断是“中断到了，所以数据已经读好，可以把线程改为 Ready”。

这条路径为什么不可靠？因为 IRQ 只说明存在待处理的设备事件。驱动仍需要读取与本 request 对应的完成证据。若完成项报告校验错误、短传输或设备失败，那么软件状态应进入错误处理，而不是伪造成功结果并唤醒为“读取成功”。

这个母例同时校准三层：

- **硬件层**：IRQ 是通知入口，不是成功值；
- **Bridge 层**：completion evidence 必须被驱动核验；
- **OS 层**：只有软件完成语义成立后，才决定唤醒、错误返回或其他后续动作。

只改变一个条件即可做微扰检查：若设备没有发 IRQ，但驱动轮询完成队列发现了成功项，请求仍然可以完成。这说明“中断”不是完成语义的必要组成部分；**可核验的完成证据**才是本 Bridge 的稳定接口对象。

11 一页压缩：用五问重建整座桥

一句话

设备/DMA 负责把请求推进到一个可观察的硬件完成状态；中断或轮询只负责让软件发现这份证据；驱动核验以后，OS 才能把硬件完成翻译成 request completion、唤醒或异步交付。

离开本册前，只需要能重新回答五问：

1. OS 提交给硬件的到底是什么状态？
2. 设备在传输期间凭什么合法访问目标缓冲区？
3. 哪个对象才是真正的“完成证据”？
4. 中断、轮询和完成证据分别处在哪一层？
5. 为什么 I/O completion 只能直接推出 Ready，而不能直接推出 Running？